DEPARTMENT OF INFORMATION TECHNOLOGY

SMT. PARMESHWARIDEVI DURGADUTT TIBREWALA

LIONS JUHU COLLEGE

OF ARTS, COMMERCE AND SCIENCE

Affiliated to University of Mumbai

(Autonomous)

J.B. NAGAR, ANDHERI (E), MUMBAI-400059

Academic Year 2025-2026

BIG DATA ANALYTICS

For

Semester II

Submitted By:

Mr. Mohd Sahbaz Chaudhary

Msc.IT (Sem II)

SMT. PARMESHWARIDEVI DURGADUTT TIBREWALA

LIONS JUHU COLLEGE

OF ARTS, COMMERCE AND SCIENCE

Affiliated to University of Mumbai

(Autonomous)

J.B. NAGAR, ANDHERI (E), MUMBAI-400059

DEPARTMENT OF INFORMATION TECHNOLOGY

Certificate of Approval

This is to certify that practical entitled **"BIG DATA ANALYTICS"**,Undertaken at **SMT.PARMESHWARIDEVI DURGADUTT TIBREWALA LIONS JUHU COLLEGE OF ARTS, COMMERCE & SCIENCE.**
By **Mr. Mohd Sahbaz Chaudhary Seat No. MIT251001** in partial fulfilment of **M.Sc. (IT) master degree (Semester II)** Examination had not been submitted for any other examination and does not form of any other course undergone by the candidate. It is further certified that he has completed all required phases of the practical.

Internal Examiner **External Examiner**

HOD / In-Charge / Coordinator

Signature/

Principal/Stamp

INDEX

Practical no. 01

Aim: Install, configure and run Hadoop and HDFS and explore HDFS

Windows Code:

Steps to Install Hadoop

1. Install Java JDK 1.8
2. Download Hadoop and extract and place under C drive
3. Set Path in Environment Variables
4. Config files under Hadoop directory
5. Create folder datanode and namenode under data directory
6. Edit HDFS and YARN files
7. Set Java Home environment in Hadoop environment
8. Setup Complete. Test by executing start-all.cmd

There are two ways to install Hadoop, i.e.

9. Single node
10. Multi node

Here, we use multi node cluster.

1. Install Java

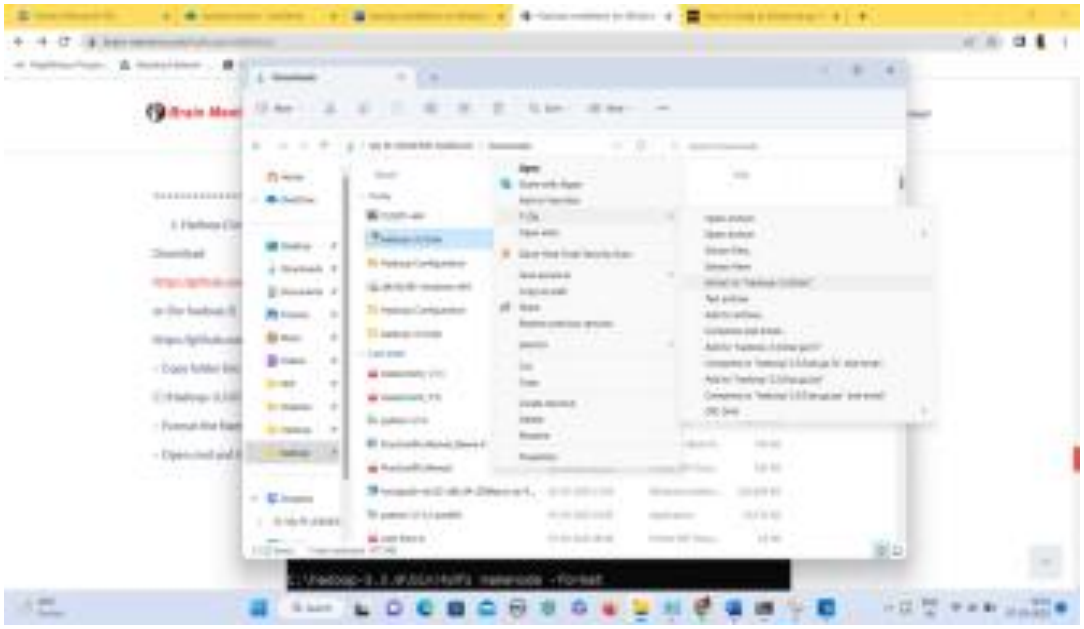
11. – Java JDK Link to download https://www.oracle.com/java/technologies/javase-jdk8-downloads.html 12. – extract and install Java in C:\Java
13. – open cmd and type -> javac -version

```
C:\Users>cd Beena

C:\Users\Beena>java -version
java version "1.8.0_361"
Java(TM) SE Runtime Environment (build 1.8.0_361-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.361-b09, mixed mode)
```

2. Download Hadoop

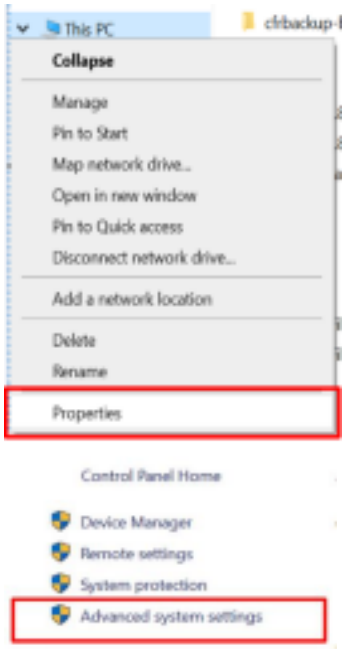
https://www.apache.org/dyn/closer.cgi/hadoop/common/hadoop-3.3.0/hadoop 3.3.0.tar.gz

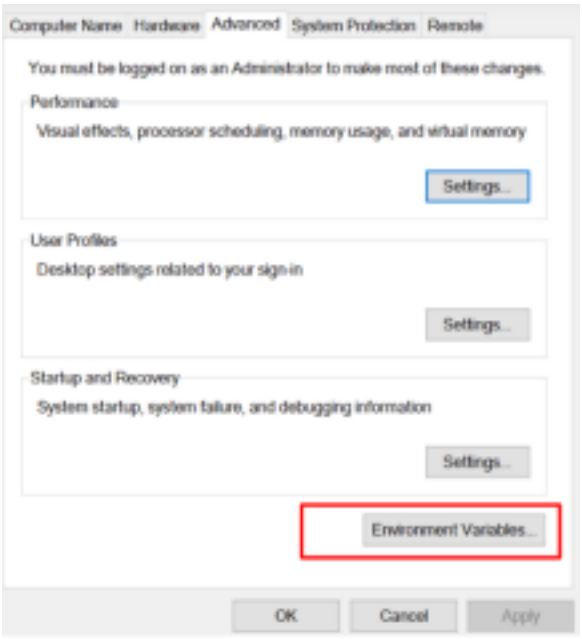
- right click .rar.gz file -> show more options -> 7-zip->and extract to C:\Hadoop 3.3.0\



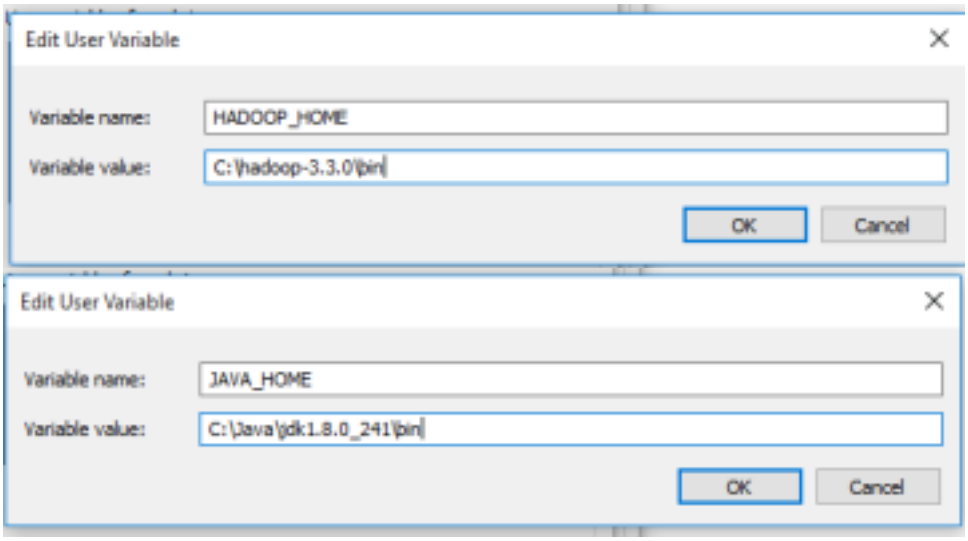
3. Set the path JAVA_HOME Environment variable

4. Set the path HADOOP_HOME Environment variable

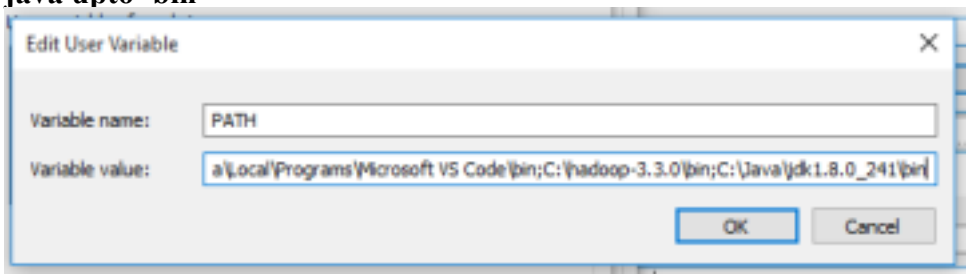




Click on **New to both user variables and system variables.**



Click on user variable -> path -> edit-> add path for Hadoop and java upto 'bin'



Click Ok, Ok, Ok.

5. Configurations
Edit file C:/Hadoop-3.3.0/etc/hadoop/core-site.xml,

paste the xml code in folder and save

==========

```
<configuration>
<property>
 <name>fs.defaultFS</name>
 <value>hdfs://localhost:9000</value>
 </property>
</configuration>
```

==========

Rename "mapred-site.xml.template" to "mapred-site.xml" and edit this file C:/Hadoop 3.3.0/etc/hadoop/mapred-site.xml, paste xml code and save this file.

==========

```
<configuration>
 <property>
 <name>mapreduce.framework.name</name>
 <value>yarn</value>
 </property>
</configuration>
```

==========

Create folder "data" under "C:\Hadoop-3.3.0"
Create folder "datanode" under "C:\Hadoop-3.3.0\data"
Create folder "namenode" under "C:\Hadoop-3.3.0\data"

==========

Edit file C:\Hadoop-3.3.0/etc/hadoop/hdfs-site.xml,

paste xml code and save this file.

```
<configuration>
<property>
 <name>dfs.replication</name>
 <value>1</value>
 </property>

 <property>
 <name>dfs.namenode.name.dir</name>
 <value>/hadoop-3.3.0/data/namenode</value>
 </property>
```

```
 <property>
 <name>dfs.datanode.data.dir</name>
<value>/hadoop-3.3.0/data/datanode</value>
 </property>
 </configuration>
```

==========

Edit file C:/Hadoop-3.3.0/etc/hadoop/yarn-site.xml,
paste xml code and save this file.

```
<configuration>
 <property>
 <name>yarn.nodemanager.aux-services</name>
 <value>mapreduce_shuffle</value>
 </property>

 <property>

<name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</n
ame>
<value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
 <property>

<name>yarn.resourcemanager.address</name>
<value>127.0.0.1:8032</value>
 </property>
 <property>

<name>yarn.resourcemanager.scheduler.address</name>
<value>127.0.0.1:8030</value>
 </property>
 <property>
 <name>yarn.resourcemanager.resource-
tracker.address</name>  <value>127.0.0.1:8031</value>
 </property>

</configuration>
```

==========

6. **Edit file C:/Hadoop-3.3.0/etc/hadoop/hadoop-env.cmd**

Find “JAVA_HOME=%JAVA_HOME%” and replace it as

set JAVA_HOME="C:\Java\jdk1.8.0_361"

==========

7. Download "redistributable" package

Download and run VC_redist.x64.exe

This is a "redistributable" package of the Visual C runtime code for 64-bit applications, from Microsoft. It contains certain shared code that every application written with Visual C expects to have available on the Windows computer it runs on.

8. Hadoop Configurations

Download bin folder from

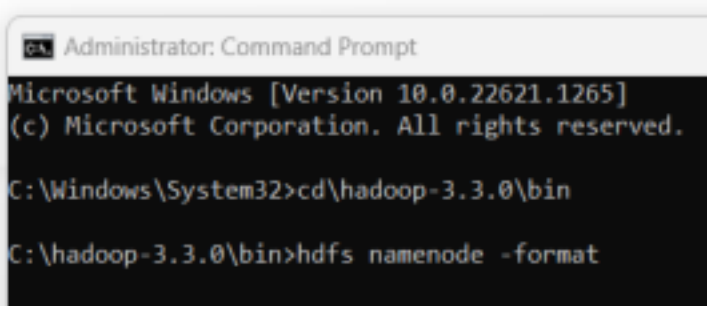
https://github.com/s911415/apache-hadoop-3.1.0-winutils

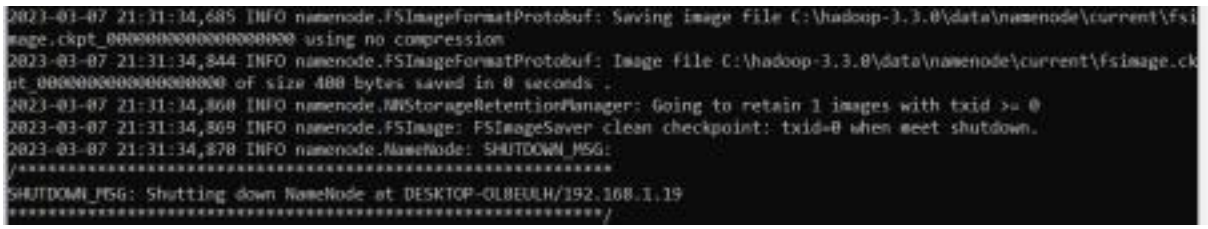
– Copy the bin folder to c:\hadoop-3.3.0. Replace the existing bin folder.

9. copy "hadoop-yarn-server-timelineservice-3.0.3.jar" from ~\hadoop 3.0.3\share\hadoop\yarn\timelineservice to ~\hadoop 3.0.3\share\hadoop\yarn folder.

10. Format the NameNode

– Open cmd 'Run as Administrator' and type command "hdfs namenode –format"





11. Testing

– Open cmd '**Run as Administrator**' and change directory to

C:\Hadoop-3.3.0\sbin – type **start-all.cmd**

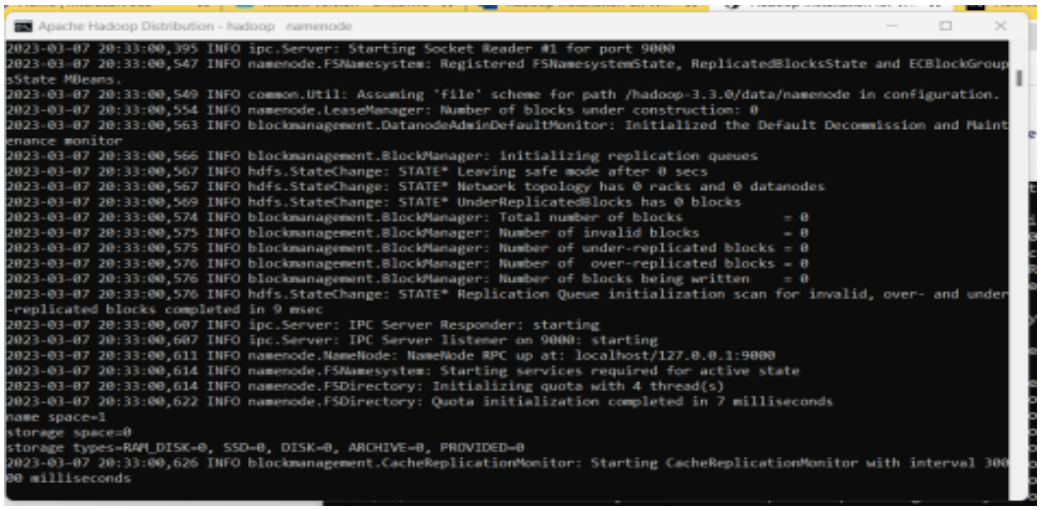
OR

- type **start-dfs.cmd**

– type **start-yarn.cmd**

```
C:\hadoop-3.3.0\sbin>start-all.cmd
This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd
The filename, directory name, or volume label syntax is incorrect.
The filename, directory name, or volume label syntax is incorrect.
starting yarn daemons
The filename, directory name, or volume label syntax is incorrect.
```

– You will get 4 more running threads for Datanode, namenode, resouce manager and node manager



Output:

12. Type JPS command to start-all.cmd command prompt, you will get following output.

```
C:\hadoop-3.3.0\sbin>jps
5632 Jps
7572 DataNode
3752 ResourceManager
7992 NameNode
8028 NodeManager
```

13. Run http://localhost:9870/ from any browser

Practical no.02

Aim: Implement word count /frequency programs using MapReduce.

Code:-

```
# Install PySpark
!pip install pyspark

# Import libraries
from pyspark.sql import SparkSession
from pyspark import SparkContext

# Initialize Spark
spark = SparkSession.builder.master("local[*]").appName("WordCount").getOrCreate()
sc = SparkContext.getOrCreate()

# Load file
words_rdd = sc.textFile("/content/sample_data/input.txt")

# Word count pipeline
word_pairs = words_rdd.flatMap(lambda line: line.split(" ")) \
                .map(lambda word: (word, 1))

# Total word count
total_words = word_pairs.count()
print("\n========== RESULT ==========")
print(f"\nTotal Words: {total_words}")

# Word frequency (reduce)
word_counts = word_pairs.reduceByKey(lambda a, b: a + b)

# Distinct words count
distinct_count = word_counts.count()
print(f"\nDistinct Words: {distinct_count}")

# Sort by frequency
sorted_counts = word_counts.map(lambda x: (x[1], x[0])) \
                    .sortByKey()

# Top 5 words
print("\nTop 5 words:")
for count, word in sorted_counts.top(5):
   print(f"{word} : {count}")
```

```
# Top 20 most frequent words
print("\nTop 20 words:")
for count, word in sorted_counts.top(20):
    print(f"{word} : {count}")

# Optional: print all sorted results (nicely formatted)
# print("\nAll words sorted:")
# for count, word in sorted_counts.collect():
#     print(f"{word} : {count}")
```

Output :-

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)

========== RESULT ==========

Total Words: 165

Distinct Words: 112

Top 5 words:
and : 9
Python : 9
as : 6
in : 5
was : 4

Top 20 words:
and : 9
Python : 9
as : 6
in : 5
was : 4
is : 4
a : 4
to : 3
the : 3
released : 3
programming : 3
language : 3
its : 3
code : 3
with : 2
on : 2
of : 2
object-oriented : 2
not : 2
write : 1
```

Practical no.03

Aim:- Implement the program using Pig

Code:-

```
# Install PySpark
!pip install pyspark

# Import required libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("Real Estate Analysis") \
    .getOrCreate()
print("\n========== DATA LOADING ==========\n")
# Load dataset (make sure file is uploaded in Colab)
data = spark.read.option("header", "true") \
            .csv("/content/sample_data/Real estate (1).csv", inferSchema=True)
# Show schema
print("Schema:\n")
data.printSchema()
# Show sample data
print("\nSample Data:\n")
data.show(10)
print("\n========== FILTERING ==========\n")
# Filter: house age > 20
filtered_data = data.filter(col("X2 house age") > 20)
print("Filtered Data (House Age > 20):\n")
filtered_data.show(10)
print("\n========== GROUPING & ANALYSIS ==========\n")
# Group by convenience stores and calculate average price
grouped_data = filtered_data.groupBy("X4 number of convenience stores")
                            .agg(avg("Y house price of unit area").alias("avg_house_price"))
# Show grouped result
print("Average House Price by Convenience Stores:\n")
grouped_data.show()
print("\n========== COMPLETED ==========\n")
# Stop Spark session
spark.stop()
```

Output:-

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)

========== DATA LOADING ==========

Schema:

root
 |-- No: integer (nullable = true)
 |-- X1 transaction date: double (nullable = true)
 |-- X2 house age: double (nullable = true)
 |-- X3 distance to the nearest MRT station: double (nullable = true)
 |-- X4 number of convenience stores: integer (nullable = true)
 |-- X5 latitude: double (nullable = true)
 |-- X6 longitude: double (nullable = true)
 |-- Y house price of unit area: double (nullable = true)


Sample Data:

+---+------------------+-----------+----------------------------------+----------------------------+----------+-----------+------------------------+
| No|X1 transaction date|X2 house age|X3 distance to the nearest MRT station|X4 number of convenience stores|X5 latitude|X6 longitude|Y house price of unit area|
+---+------------------+-----------+----------------------------------+----------------------------+----------+-----------+------------------------+
|  1|          2012.917|       32.0|                          84.87882|                          10|  24.98298|  121.54024|                    37.9|
|  2|          2012.917|       19.5|                          306.5947|                           9|  24.98034|  121.53951|                    42.2|
|  3|          2013.583|       13.3|                          561.9845|                           5|  24.98746|  121.54391|                    47.3|
|  4|            2013.5|       13.3|                          561.9845|                           5|  24.98746|  121.54391|                    54.8|
|  5|          2012.833|        5.0|                          390.5684|                           5|  24.97937|  121.54245|                    43.1|
|  6|          2012.667|        7.1|                           2175.03|                           3|  24.96305|  121.51254|                    32.1|
|  7|          2012.667|       34.5|                          623.4731|                           7|  24.97933|  121.53642|                    40.3|
|  8|          2013.417|       20.3|                          287.6025|                           6|  24.98042|  121.54228|                    46.7|
|  9|            2013.5|       31.7|                          5512.038|                           1|  24.95095|  121.48458|                    18.8|
| 10|          2013.417|       17.9|                           1783.18|                           3|  24.96731|  121.51486|                    22.1|
+---+------------------+-----------+----------------------------------+----------------------------+----------+-----------+------------------------+
only showing top 10 rows

========== FILTERING ==========

Filtered Data (House Age > 20):

+---+------------------+-----------+----------------------------------+----------------------------+----------+-----------+------------------------+
| No|X1 transaction date|X2 house age|X3 distance to the nearest MRT station|X4 number of convenience stores|X5 latitude|X6 longitude|Y house price of unit area|
+---+------------------+-----------+----------------------------------+----------------------------+----------+-----------+------------------------+
|  1|          2012.917|       32.0|                          84.87882|                          10|  24.98298|  121.54024|                    37.9|
|  7|          2012.667|       34.5|                          623.4731|                           7|  24.97933|  121.53642|                    40.3|
|  8|          2013.417|       20.3|                          287.6025|                           6|  24.98042|  121.54228|                    46.7|
|  9|            2013.5|       31.7|                          5512.038|                           1|  24.95095|  121.48458|                    18.8|
| 11|          2013.083|       34.8|                          405.2134|                           1|  24.97349|  121.53372|                    41.4|
| 14|          2012.667|       20.4|                          2469.645|                           4|  24.96108|  121.51046|                    23.8|
| 16|          2013.583|       35.7|                          579.2083|                           2|   24.9824|  121.54619|                    50.5|
| 25|            2013.0|       39.6|                          480.6977|                           4|  24.97353|  121.53885|                    38.8|
| 26|          2013.083|       29.3|                          1487.868|                           2|  24.97542|  121.51726|                    27.0|
| 31|            2013.5|       25.9|                           4519.69|                           0|  24.94826|  121.49587|                    22.1|
+---+------------------+-----------+----------------------------------+----------------------------+----------+-----------+------------------------+
only showing top 10 rows

========== GROUPING & ANALYSIS ==========

Average House Price by Convenience Stores:

+----------------------------+------------------+
|X4 number of convenience stores|   avg_house_price|
+----------------------------+------------------+
|                           1|24.516666666666666|
|                           6| 40.25714285714286|
|                           3|           33.1125|
|                           5|              34.2|
|                           9| 48.41250000000001|
|                           4| 37.50909090909091|
|                           8|44.605000000000004|
|                           7| 38.51666666666667|
|                          10| 45.26666666666667|
|                           2| 34.82222222222222|
|                           0|23.060000000000002|
+----------------------------+------------------+


========== COMPLETED ==========
```

Practical no.04

Aim:- Implement the application in Hive.

Code:-

```
# ========================================
# INSTALL PYSPARK
# ========================================
!pip install pyspark

# ========================================
# IMPORT LIBRARIES
# ========================================
import os
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import col, avg
from pyspark.sql import Row
from pyspark import SparkContext, SparkConf

# ========================================
# START SPARK WITH HIVE SUPPORT
# ========================================
spark = SparkSession.builder \
    .appName("Spark Hive Example") \
    .enableHiveSupport() \
    .getOrCreate()

sc = SparkContext.getOrCreate()

print("\n========== DATABASE INFO ==========\n")

spark.sql("SHOW DATABASES").show()
spark.sql("SHOW TABLES").show()

# ========================================
# SHOW FUNCTIONS
# ========================================
print("\n========== SAMPLE FUNCTIONS ==========\n")
fncs = spark.sql("SHOW FUNCTIONS").collect()
for i in fncs[100:110]:
    print(i[0])
spark.sql("DESCRIBE FUNCTION instr").show(truncate=False)
```

```
# ========================================
# CREATE DATABASE
# ========================================
print("\n========== CREATE DATABASE ==========\n")
spark.sql("CREATE DATABASE IF NOT EXISTS movies")
spark.sql("USE movies")
spark.sql("SHOW DATABASES").show()

# ========================================
# DOWNLOAD DATASET
# ========================================
print("\n========== DOWNLOAD DATA ==========\n")
!wget http://files.grouplens.org/datasets/movielens/ml-latest.zip
!unzip -o /content/ml-latest.zip

# ========================================
# CREATE TABLES
# ========================================
print("\n========== CREATE TABLES ==========\n")

spark.sql("""
CREATE TABLE IF NOT EXISTS movies (
    movieId INT,
    title STRING,
    genres STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
""")

spark.sql("""
CREATE TABLE IF NOT EXISTS ratings (
    userId INT,
    movieId INT,
    rating FLOAT,
    timestamp STRING
)
STORED AS ORC
""")

spark.sql("""
CREATE TABLE IF NOT EXISTS genres_by_count (
    genres STRING,
```

```
    count INT
)
STORED AS AVRO
""")

spark.sql("SHOW TABLES").show()

# ========================================
# LOAD DATA INTO MOVIES TABLE
# ========================================
print("\n========== LOAD MOVIES DATA ==========\n")

spark.sql("""
LOAD DATA LOCAL INPATH '/content/ml-latest/movies.csv'
OVERWRITE INTO TABLE movies
""")

spark.sql("SELECT * FROM movies LIMIT 10").show(truncate=False)

# ========================================
# READ RATINGS DATA (DATAFRAME)
# ========================================
print("\n========== READ RATINGS ==========\n")

schema = StructType([
    StructField('userId', IntegerType()),
    StructField('movieId', IntegerType()),
    StructField('rating', DoubleType()),
    StructField('timestamp', StringType())
])

ratings_df = spark.read.csv(
    "/content/ml-latest/ratings.csv",
    schema=schema,
    header=True
)

ratings_df.show(5)
ratings_df.printSchema()
```

```
# ========================================
# RDD → DATAFRAME METHOD
# ========================================
print("\n========== RDD TO DATAFRAME ==========\n")
rdd = sc.textFile("/content/ml-latest/ratings.csv")
header = rdd.first()

ratings_df2 = rdd.filter(lambda x: x != header) \
    .map(lambda x: Row(
        userId=int(x.split(",")[0]),
        movieId=int(x.split(",")[1]),
        rating=float(x.split(",")[2]),
        timestamp=x.split(",")[3]
    )).toDF()

ratings_df2.show(5)

# ========================================
# ANALYSIS: GENRE COUNT
# ========================================
print("\n========== GENRE ANALYSIS ==========\n")

spark.sql("""
SELECT genres, COUNT(*) AS count
FROM movies
GROUP BY genres
HAVING COUNT(*) > 500
ORDER BY count DESC
""").show()

# ========================================
# INSERT INTO AVRO TABLE
# ========================================
print("\n========== INSERT INTO AVRO TABLE ==========\n")

spark.sql("""
INSERT INTO genres_by_count
SELECT genres, COUNT(*) AS count
FROM movies
GROUP BY genres
HAVING COUNT(*) >= 500
ORDER BY count DESC
""")
```

```
spark.sql("""
SELECT * FROM genres_by_count
ORDER BY count DESC
LIMIT 3
""").show()

# ========================================
# TAGS DATAFRAME + TEMP TABLE
# ========================================
print("\n========== TAGS TEMP TABLE ==========\n")

tags_schema = StructType([
    StructField('userId', IntegerType()),
    StructField('movieId', IntegerType()),
    StructField('tag', StringType()),
    StructField('timestamp', StringType())
])

tags_df = spark.read.csv(
    "/content/ml-latest/tags.csv",
    schema=tags_schema,
    header=True
)

tags_df.printSchema()

# Register temp table
tags_df.createOrReplaceTempView("tags_df_table")

spark.sql("SHOW TABLES").show()

# ========================================
# STOP SPARK
# ========================================
print("\n========== COMPLETED ==========\n")
spark.stop()
```

Output:-

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)

========== DATABASE INFO ==========

+---------+
|namespace|
+---------+
|  default|
+---------+

+---------+---------+-----------+
|namespace|tableName|isTemporary|
+---------+---------+-----------+
+---------+---------+-----------+


========== SAMPLE FUNCTIONS ==========

contains
conv
convert_timezone
corr
cos
cosh
cot
count
count_if
count_min_sketch
+----------------------------------------------------------------------------------------------------+
|function_desc                                                                                       |
+----------------------------------------------------------------------------------------------------+
|Function: instr                                                                                     |
|Class: org.apache.spark.sql.catalyst.expressions.StringInstr                                        |
|Usage: instr(str, substr) - Returns the (1-based) index of the first occurrence of `substr` in `str`.|
+----------------------------------------------------------------------------------------------------+


========== CREATE DATABASE ==========

+---------+
|namespace|
+---------+
|  default|
|   movies|
+---------+
```

```
========== CREATE DATABASE ==========

+---------+
|namespace|
+---------+
|  default|
|   movies|
+---------+


========== DOWNLOAD DATA ==========

--2026-04-11 02:38:27--  http://files.grouplens.org/datasets/movielens/ml-latest.zip
Resolving files.grouplens.org (files.grouplens.org)... 128.101.96.204
Connecting to files.grouplens.org (files.grouplens.org)|128.101.96.204|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://files.grouplens.org/datasets/movielens/ml-latest.zip [following]
--2026-04-11 02:38:27--  https://files.grouplens.org/datasets/movielens/ml-latest.zip
Connecting to files.grouplens.org (files.grouplens.org)|128.101.96.204|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 350896731 (335M) [application/zip]
Saving to: ‘ml-latest.zip’

ml-latest.zip       100%[===================>] 334.64M  16.9MB/s    in 18s

2026-04-11 02:38:45 (19.0 MB/s) - ‘ml-latest.zip’ saved [350896731/350896731]

Archive:  /content/ml-latest.zip
   creating: ml-latest/
  inflating: ml-latest/tags.csv
  inflating: ml-latest/links.csv
  inflating: ml-latest/README.txt
  inflating: ml-latest/ratings.csv
  inflating: ml-latest/genome-tags.csv
  inflating: ml-latest/genome-scores.csv
  inflating: ml-latest/movies.csv

========== CREATE TABLES ==========

+---------+---------------+-----------+
|namespace|      tableName|isTemporary|
+---------+---------------+-----------+
|   movies|genres_by_count|      false|
|   movies|         movies|      false|
|   movies|        ratings|      false|
+---------+---------------+-----------+
```

```
========== LOAD MOVIES DATA ==========

+-------+---------------------------------+-----------------------------------------+
|movieId|title                            |genres                                   |
+-------+---------------------------------+-----------------------------------------+
|NULL   |title                            |genres                                   |
|1      |Toy Story (1995)                 |Adventure|Animation|Children|Comedy|Fantasy|
|2      |Jumanji (1995)                   |Adventure|Children|Fantasy               |
|3      |Grumpier Old Men (1995)          |Comedy|Romance                           |
|4      |Waiting to Exhale (1995)         |Comedy|Drama|Romance                     |
|5      |Father of the Bride Part II (1995)|Comedy                                  |
|6      |Heat (1995)                      |Action|Crime|Thriller                    |
|7      |Sabrina (1995)                   |Comedy|Romance                           |
|8      |Tom and Huck (1995)              |Adventure|Children                       |
|9      |Sudden Death (1995)              |Action                                   |
+-------+---------------------------------+-----------------------------------------+


========== READ RATINGS ==========

+------+-------+------+----------+
|userId|movieId|rating| timestamp|
+------+-------+------+----------+
|     1|      1|   4.0|1225734739|
|     1|    110|   4.0|1225865086|
|     1|    158|   4.0|1225733503|
|     1|    260|   4.5|1225735204|
|     1|    356|   5.0|1225735119|
+------+-------+------+----------+
only showing top 5 rows
root
 |-- userId: integer (nullable = true)
 |-- movieId: integer (nullable = true)
 |-- rating: double (nullable = true)
 |-- timestamp: string (nullable = true)


========== RDD TO DATAFRAME ==========

+------+-------+------+----------+
|userId|movieId|rating| timestamp|
+------+-------+------+----------+
|     1|      1|   4.0|1225734739|
|     1|    110|   4.0|1225865086|
|     1|    158|   4.0|1225733503|
|     1|    260|   4.5|1225735204|
|     1|    356|   5.0|1225735119|
+------+-------+------+----------+
only showing top 5 rows
```

```
========== GENRE ANALYSIS ==========

+--------------------+-----+
|              genres|count|
+--------------------+-----+
|               Drama|10902|
|         Documentary| 7493|
|              Comedy| 6988|
|  (no genres listed)| 6843|
|        Comedy|Drama| 2855|
|       Drama|Romance| 2504|
|              Horror| 2318|
|      Comedy|Romance| 1976|
|            Thriller| 1318|
|Comedy|Drama|Romance| 1158|
|     Horror|Thriller| 1138|
|      Drama|Thriller| 1133|
|           Animation| 1085|
|         Crime|Drama| 1031|
|              Action|  745|
|           Drama|War|  698|
|        Action|Drama|  654|
|             Western|  608|
|             Romance|  587|
|Crime|Drama|Thriller|  581|
+--------------------+-----+
only showing top 20 rows

========== INSERT INTO AVRO TABLE ==========

+-----------+-----+
|     genres|count|
+-----------+-----+
|      Drama|10902|
|Documentary| 7493|
|     Comedy| 6988|
+-----------+-----+


========== TAGS TEMP TABLE ==========

root
 |-- userId: integer (nullable = true)
 |-- movieId: integer (nullable = true)
 |-- tag: string (nullable = true)
 |-- timestamp: string (nullable = true)

+---------+---------------+-----------+
|namespace|      tableName|isTemporary|
+---------+---------------+-----------+
|   movies|genres_by_count|      false|
|   movies|         movies|      false|
|   movies|        ratings|      false|
|         |  tags_df_table|       true|
+---------+---------------+-----------+


========== COMPLETED ==========
```

Practical no.05

Aim :- Implement Decision tree classification techniques

Code:-

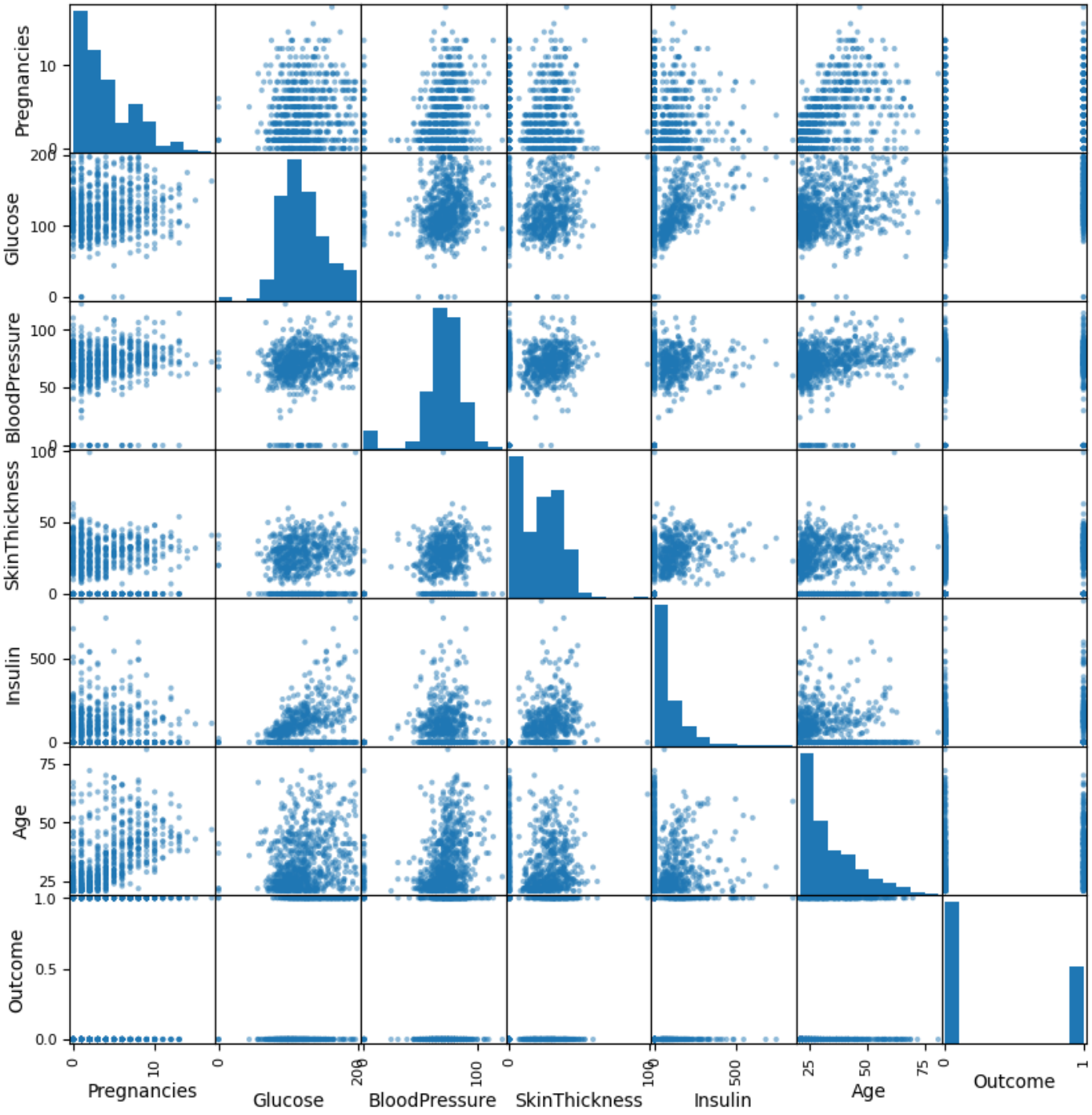
```
# ================================
# 1. Install PySpark
# ================================
!pip install pyspark

# ================================
# 2. Import Libraries
# ================================
from pyspark.sql import SparkSession
import pandas as pd

# ================================
# 3. Create Spark Session
# ================================
spark = SparkSession.builder.appName('ml-diabetes').getOrCreate()

# ================================
# 4. Load Dataset
# (Upload diabetes.csv manually in Colab first)
# ================================
df = spark.read.csv('/content/sample_data/diabetes.csv', header=True, inferSchema=True)

# ================================
# 5. Basic Info
# ================================
df.printSchema()
df.show(5)

# ================================
# 6. Convert to Pandas (for analysis)
# ================================
pdf = df.toPandas()
print(pdf.head())

# ================================
# 7. GroupBy using Pandas
# ================================
print(pdf.groupby('Outcome').count())
```

```
# ==================================
# 8. Describe Data
# ==================================
numeric_features = [t[0] for t in df.dtypes if t[1] == 'int']
desc = df.select(numeric_features).describe().toPandas()
print(desc.transpose())


# ==================================
# 9. Visualization (Pandas)
# ==================================
from pandas.plotting import scatter_matrix
import matplotlib.pyplot as plt
numeric_data = df.select(numeric_features).toPandas()
scatter_matrix(numeric_data, figsize=(8, 8))
plt.show()


# ==================================
# 10. Check Missing Values
# ==================================
from pyspark.sql.functions import isnull, when, count
df.select([count(when(isnull(c), c)).alias(c) for c in df.columns]).show()


# ==================================
# 11. Feature Selection (Drop columns)
# ==================================
dataset = df.drop('SkinThickness').drop('Insulin')
dataset_new = dataset.drop('DiabetesPedigreeFunction')
dataset_final = dataset_new.drop('Pregnancies')
dataset_final.show(5)


# ==================================
# 12. Feature Engineering
# ==================================
from pyspark.ml.feature import VectorAssembler
required_features = ['Glucose','BloodPressure','BMI','Age']
assembler = VectorAssembler(inputCols=required_features, outputCol='features')
transformed_data = assembler.transform(dataset_final)
transformed_data.select('features','Outcome').show(5)
```

```
# ==================================
# 13. Train-Test Split
# ==================================
(training_data, test_data) = transformed_data.randomSplit([0.8,0.2], seed=2020)
print("Training Count:", training_data.count())
print("Test Count:", test_data.count())

# ==================================
# 14. Decision Tree Model
# ==================================
from pyspark.ml.classification import DecisionTreeClassifier
dt = DecisionTreeClassifier(featuresCol='features', labelCol='Outcome', maxDepth=3)
dt_model = dt.fit(training_data)
dt_predictions = dt_model.transform(test_data)
dt_predictions.select('features','Outcome','prediction').show(5)

# ==================================
# 15. Evaluate Decision Tree
# ==================================
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
evaluator = MulticlassClassificationEvaluator(labelCol='Outcome', metricName='accuracy')
dt_accuracy = evaluator.evaluate(dt_predictions)
print("Decision Tree Accuracy:", dt_accuracy)

# ==================================
# 16. Gradient Boosted Trees
# ==================================
from pyspark.ml.classification import GBTClassifier
gb = GBTClassifier(labelCol='Outcome', featuresCol='features')
gb_model = gb.fit(training_data)
gb_predictions = gb_model.transform(test_data)

# ==================================
# 17. Evaluate GBT
# ==================================
gb_accuracy = evaluator.evaluate(gb_predictions)
print("Gradient Boosted Trees Accuracy:", gb_accuracy)
```

Output:-

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
root
 |-- Pregnancies: integer (nullable = true)
 |-- Glucose: integer (nullable = true)
 |-- BloodPressure: integer (nullable = true)
 |-- SkinThickness: integer (nullable = true)
 |-- Insulin: integer (nullable = true)
 |-- BMI: double (nullable = true)
 |-- DiabetesPedigreeFunction: double (nullable = true)
 |-- Age: integer (nullable = true)
 |-- Outcome: integer (nullable = true)

+-----------+-------+-------------+-------------+-------+----+------------------------+---+-------+
|Pregnancies|Glucose|BloodPressure|SkinThickness|Insulin| BMI|DiabetesPedigreeFunction|Age|Outcome|
+-----------+-------+-------------+-------------+-------+----+------------------------+---+-------+
|          6|    148|           72|           35|      0|33.6|                   0.627| 50|      1|
|          1|     85|           66|           29|      0|26.6|                   0.351| 31|      0|
|          8|    183|           64|            0|      0|23.3|                   0.672| 32|      1|
|          1|     89|           66|           23|     94|28.1|                   0.167| 21|      0|
|          0|    137|           40|           35|    168|43.1|                   2.288| 33|      1|
+-----------+-------+-------------+-------------+-------+----+------------------------+---+-------+
only showing top 5 rows
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
         Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  BMI  \
Outcome
0                500      500            500            500      500  500
1                268      268            268            268      268  268

         DiabetesPedigreeFunction  Age
Outcome
0                             500  500
1                             268  268
                 0                1                2    3    4
summary        count             mean           stddev  min  max
Pregnancies      768  3.8450520833333335   3.36957806269887    0   17
Glucose          768       120.89453125  31.97261819513622    0  199
BloodPressure    768        69.10546875  19.35580717064477    0  122
SkinThickness    768  20.536458333333332  15.952217567727642    0   99
Insulin          768   79.79947916666667  115.24400235133803    0  846
Age              768  33.240885416666664  11.760231540678689   21   81
Outcome          768  0.3489583333333333   0.476951377242799    0    1
```


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
Age
Outcome

```
+-----------+-------+-------------+-------------+-------+---+-----------------------+---+-------+
|Pregnancies|Glucose|BloodPressure|SkinThickness|Insulin|BMI|DiabetesPedigreeFunction|Age|Outcome|
+-----------+-------+-------------+-------------+-------+---+-----------------------+---+-------+
|          0|      0|            0|            0|      0|  0|                      0|  0|      0|
+-----------+-------+-------------+-------------+-------+---+-----------------------+---+-------+

+-------+-------------+----+---+-------+
|Glucose|BloodPressure| BMI|Age|Outcome|
+-------+-------------+----+---+-------+
|    148|           72|33.6| 50|      1|
|     85|           66|26.6| 31|      0|
|    183|           64|23.3| 32|      1|
|     89|           66|28.1| 21|      0|
|    137|           40|43.1| 33|      1|
+-------+-------------+----+---+-------+
only showing top 5 rows
+--------------------+-------+
|            features|Outcome|
+--------------------+-------+
|[148.0,72.0,33.6,...|      1|
|[85.0,66.0,26.6,3...|      0|
|[183.0,64.0,23.3,...|      1|
|[89.0,66.0,28.1,2...|      0|
|[137.0,40.0,43.1,...|      1|
+--------------------+-------+
only showing top 5 rows
Training Count: 620
Test Count: 148
+--------------------+-------+----------+
|            features|Outcome|prediction|
+--------------------+-------+----------+
|[57.0,80.0,32.8,4...|      0|       0.0|
|[67.0,76.0,45.3,4...|      0|       0.0|
|[71.0,48.0,20.4,2...|      0|       0.0|
|[71.0,78.0,33.2,2...|      0|       0.0|
|[72.0,78.0,31.6,3...|      0|       0.0|
+--------------------+-------+----------+
only showing top 5 rows
Decision Tree Accuracy: 0.7702702702702703
Gradient Boosted Trees Accuracy: 0.7567567567567568
```

Practical no.06

Aim:- Implement using Linear_Regression using pyspark

Code:-

```
# ========================================
# ALL-IN-ONE PYSPARK LINEAR REGRESSION CODE
# ========================================

# 1. Install PySpark
!pip install pyspark

# 2. Imports
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression

# 3. Start Spark Session
spark =
SparkSession.builder.master("local").appName("linear_regression_model").getOrCreate()

# 4. Load Dataset (UPLOAD 'Real estate.csv' FIRST)
df = spark.read.csv("/content/sample_data/Real estate.csv", header=True, inferSchema=True)

# 5. Show Data
print("Schema:")
df.printSchema()

print("Sample Data:")
df.show(5)

# 6. Summary Statistics
print("Summary:")
df.describe().show()

# 7. Feature Engineering (VectorAssembler)
assembler = VectorAssembler(
   inputCols=[
      'X1 transaction date',
      'X2 house age',
      'X3 distance to the nearest MRT station',
      'X4 number of convenience stores',
      'X5 latitude',
      'X6 longitude'
```

```
    ],
    outputCol='features'
)

data = assembler.transform(df)

# Select only features + label
data = data.select('features', 'Y house price of unit area')

print("Features Data:")
data.show(5)

# 8. Train-Test Split
train_data, test_data = data.randomSplit([0.7, 0.3], seed=42)

print("Train Count:", train_data.count())
print("Test Count:", test_data.count())

# 9. Train Model
lr = LinearRegression(
    featuresCol='features',
    labelCol='Y house price of unit area'
)

model = lr.fit(train_data)

# 10. Predictions
predictions = model.transform(test_data)
print("Predictions:")
predictions.select('features', 'Y house price of unit area', 'prediction').show(5)

# 11. Evaluation
results = model.evaluate(test_data)
print("Model Evaluation:")
print("RMSE:", results.rootMeanSquaredError)
print("MSE:", results.meanSquaredError)
print("R2 Score:", results.r2)

# 12. Coefficients
print("Model Coefficients:")
print("Intercept:", model.intercept)
print("Coefficients:", model.coefficients)
print("DONE ✅ ")
```

Output:-

```
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
Schema:
root
 |-- No: integer (nullable = true)
 |-- X1 transaction date: double (nullable = true)
 |-- X2 house age: double (nullable = true)
 |-- X3 distance to the nearest MRT station: double (nullable = true)
 |-- X4 number of convenience stores: integer (nullable = true)
 |-- X5 latitude: double (nullable = true)
 |-- X6 longitude: double (nullable = true)
 |-- Y house price of unit area: double (nullable = true)

Sample Data:
+---+------------------+-----------+---------------------------------+---------------------------+----------+-----------+--------------------------+
| No|X1 transaction date|X2 house age|X3 distance to the nearest MRT station|X4 number of convenience stores|X5 latitude|X6 longitude|Y house price of unit area|
+---+------------------+-----------+---------------------------------+---------------------------+----------+-----------+--------------------------+
|  1|          2012.917|       32.0|                         84.87882|                         10|   24.98298|   121.54024|                      37.9|
|  2|          2012.917|       19.5|                         306.5947|                          9|   24.98034|   121.53951|                      42.2|
|  3|          2013.583|       13.3|                         561.9845|                          5|   24.98746|   121.54391|                      47.3|
|  4|            2013.5|       13.3|                         561.9845|                          5|   24.98746|   121.54391|                      54.8|
|  5|          2012.833|        5.0|                         390.5684|                          5|   24.97937|   121.54245|                      43.1|
+---+------------------+-----------+---------------------------------+---------------------------+----------+-----------+--------------------------+
only showing top 5 rows
Summary:
+-------+-----------------+------------------+-----------------+---------------------------------+---------------------------+-------------------+-------------------+--------------------------+
|summary|               No|X1 transaction date|      X2 house age|X3 distance to the nearest MRT station|X4 number of convenience stores|        X5 latitude|       X6 longitude|Y house price of unit area|
+-------+-----------------+------------------+-----------------+---------------------------------+---------------------------+-------------------+-------------------+--------------------------+
|  count|              414|               414|              414|                              414|                        414|                414|                414|                       414|
|   mean|            207.5| 2013.1489710144933| 17.71256038647343|                1083.8856889130436|          4.094202898550725| 24.969030072463745| 121.53336108695667|         37.98019323671498|
| stddev|119.6557562342907| 0.2819672402629999|11.392484533242524|                1262.1095954078 51|          2.945561805663 6177|0.012410196590450208|0.015347183004592374|        13.606487697735316|
|    min|                1|          2012.667|              0.0|                         23.38284|                          0|           24.93207|          121.47353|                       7.6|
|    max|              414|          2013.583|             43.8|                         6488.021|                         10|           25.01459|          121.56627|                      117.5|
+-------+-----------------+------------------+-----------------+---------------------------------+---------------------------+-------------------+-------------------+--------------------------+

Features Data:
+--------------------+--------------------------+
|            features|Y house price of unit area|
+--------------------+--------------------------+
|[2012.917,32.0,84...|                      37.9|
|[2012.917,19.5,30...|                      42.2|
|[2013.583,13.3,56...|                      47.3|
|[2013.5,13.3,561....|                      54.8|
|[2012.833,5.0,390...|                      43.1|
+--------------------+--------------------------+
only showing top 5 rows
Train Count: 310
Test Count: 104
Predictions:
+--------------------+--------------------------+------------------+
|            features|Y house price of unit area|        prediction|
+--------------------+--------------------------+------------------+
|[2012.667,1.5,23....|                      47.7| 48.14199260293026|
|[2012.667,5.6,90....|                      50.0|51.127677509066416|
|[2012.667,7.1,217...|                      32.1| 31.58847671017429|
|[2012.667,12.4,17...|                      31.3|30.629401387002872|
|[2012.667,15.5,81...|                      37.4| 39.46606291669377|
+--------------------+--------------------------+------------------+
only showing top 5 rows
Model Evaluation:
RMSE: 10.162569249787284
MSE: 103.27781375672208
R2 Score: 0.4700390683272475
Model Coefficients:
Intercept: -17461.99205896899
Coefficients: [4.524047621367476,-0.28163806671176733,-0.003515222614975291,1.4271892815130738,222.90005166067255,23.283834114485327]
DONE ✅
```

Practical no.07

Aim:- Implement using Logistic_Regression using pyspark

Code:-

```
# Install PySpark
!pip install pyspark

# Start Spark Session
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('ml-diabetes').getOrCreate()

# Load Dataset (Make sure diabetes.csv is uploaded in Colab)
df = spark.read.csv('/content/sample_data/diabetes.csv', header=True, inferSchema=True)

# Show schema and sample data
df.printSchema()
df.show(5)

# Convert small sample to pandas (for visualization)
import pandas as pd
pd.DataFrame(df.take(5), columns=df.columns).transpose()

# Check class distribution
df.groupby('Outcome').count().show()

# Select numeric features
numeric_features = [t[0] for t in df.dtypes if t[1] == 'int']
print("Numeric Features:", numeric_features)

# Statistical summary
df.select(numeric_features).describe().show()

# Check missing values
from pyspark.sql.functions import isnull, when, count

df.select([count(when(isnull(c), c)).alias(c) for c in df.columns]).show()

# Drop unnecessary columns
dataset = df.drop('SkinThickness')
dataset = dataset.drop('Insulin')
dataset = dataset.drop('DiabetesPedigreeFunction')
dataset = dataset.drop('Pregnancies')
```

```
dataset.show(5)

# Feature Engineering - Convert into vector
from pyspark.ml.feature import VectorAssembler

required_features = ['Glucose', 'BloodPressure', 'BMI', 'Age']

assembler = VectorAssembler(inputCols=required_features, outputCol='features')
transformed_data = assembler.transform(dataset)

transformed_data.select('features', 'Outcome').show(5)

# Train-Test Split
(training_data, test_data) = transformed_data.randomSplit([0.8, 0.2], seed=2020)

print("Training count:", training_data.count())
print("Test count:", test_data.count())

# Logistic Regression Model
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(featuresCol='features', labelCol='Outcome', maxIter=10)
lrModel = lr.fit(training_data)

# Predictions
predictions = lrModel.transform(test_data)

predictions.select('features', 'Outcome', 'prediction').show(10)

# Model Evaluation
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

evaluator = MulticlassClassificationEvaluator(labelCol='Outcome', metricName='accuracy')

accuracy = evaluator.evaluate(predictions)

print("✅ Logistic Regression Accuracy:", accuracy)
```

Output:-

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
root
 |-- Pregnancies: integer (nullable = true)
 |-- Glucose: integer (nullable = true)
 |-- BloodPressure: integer (nullable = true)
 |-- SkinThickness: integer (nullable = true)
 |-- Insulin: integer (nullable = true)
 |-- BMI: double (nullable = true)
 |-- DiabetesPedigreeFunction: double (nullable = true)
 |-- Age: integer (nullable = true)
 |-- Outcome: integer (nullable = true)

+-----------+-------+-------------+-------------+-------+----+------------------------+---+-------+
|Pregnancies|Glucose|BloodPressure|SkinThickness|Insulin| BMI|DiabetesPedigreeFunction|Age|Outcome|
+-----------+-------+-------------+-------------+-------+----+------------------------+---+-------+
|          6|    148|           72|           35|      0|33.6|                   0.627| 50|      1|
|          1|     85|           66|           29|      0|26.6|                   0.351| 31|      0|
|          8|    183|           64|            0|      0|23.3|                   0.672| 32|      1|
|          1|     89|           66|           23|     94|28.1|                   0.167| 21|      0|
|          0|    137|           40|           35|    168|43.1|                   2.288| 33|      1|
+-----------+-------+-------------+-------------+-------+----+------------------------+---+-------+
only showing top 5 rows
+-------+-----+
|Outcome|count|
+-------+-----+
|      1|  268|
|      0|  500|
+-------+-----+

Numeric Features: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'Age', 'Outcome']
+-------+------------------+-----------------+-----------------+-----------------+-----------------+-----------------+-------------------+
|summary|       Pregnancies|          Glucose|    BloodPressure|    SkinThickness|          Insulin|              Age|            Outcome|
+-------+------------------+-----------------+-----------------+-----------------+-----------------+-----------------+-------------------+
|  count|               768|              768|              768|              768|              768|              768|                768|
|   mean|3.8450520833333335|    120.89453125|     69.10546875|20.536458333333332|79.79947916666667|33.240885416666664|0.3489583333333333|
| stddev| 3.36957806269887|31.97261819513622|19.35580717064477|15.952217567727642|115.24400235133803|11.760231540678689|0.47695137724279|
|    min|                 0|                0|                0|                0|                0|               21|                  0|
|    max|                17|              199|              122|               99|              846|               81|                  1|
+-------+------------------+-----------------+-----------------+-----------------+-----------------+-----------------+-------------------+

+-----------+-------+-------------+-------------+-------+---+------------------------+---+-------+
|Pregnancies|Glucose|BloodPressure|SkinThickness|Insulin|BMI|DiabetesPedigreeFunction|Age|Outcome|
+-----------+-------+-------------+-------------+-------+---+------------------------+---+-------+
|          0|      0|            0|            0|      0|  0|                       0|  0|      0|
+-----------+-------+-------------+-------------+-------+---+------------------------+---+-------+
only showing top 5 rows
+-------+-----+
|Outcome|count|
+-------+-----+
|      1|  268|
|      0|  500|
+-------+-----+

Numeric Features: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'Age', 'Outcome']
+-------+------------------+-----------------+-----------------+-----------------+-----------------+-----------------+-------------------+
|summary|       Pregnancies|          Glucose|    BloodPressure|    SkinThickness|          Insulin|              Age|            Outcome|
+-------+------------------+-----------------+-----------------+-----------------+-----------------+-----------------+-------------------+
|  count|               768|              768|              768|              768|              768|              768|                768|
|   mean|3.8450520833333335|    120.89453125|     69.10546875|20.536458333333332|79.79947916666667|33.240885416666664|0.3489583333333333|
| stddev| 3.36957806269887|31.97261819513622|19.35580717064477|15.952217567727642|115.24400235133803|11.760231540678689|0.47695137724279|
|    min|                 0|                0|                0|                0|                0|               21|                  0|
|    max|                17|              199|              122|               99|              846|               81|                  1|
+-------+------------------+-----------------+-----------------+-----------------+-----------------+-----------------+-------------------+

+-----------+-------+-------------+-------------+-------+---+------------------------+---+-------+
|Pregnancies|Glucose|BloodPressure|SkinThickness|Insulin|BMI|DiabetesPedigreeFunction|Age|Outcome|
+-----------+-------+-------------+-------------+-------+---+------------------------+---+-------+
|          0|      0|            0|            0|      0|  0|                       0|  0|      0|
+-----------+-------+-------------+-------------+-------+---+------------------------+---+-------+

+-------+-------------+----+---+-------+
|Glucose|BloodPressure| BMI|Age|Outcome|
+-------+-------------+----+---+-------+
|    148|           72|33.6| 50|      1|
|     85|           66|26.6| 31|      0|
|    183|           64|23.3| 32|      1|
|     89|           66|28.1| 21|      0|
|    137|           40|43.1| 33|      1|
+-------+-------------+----+---+-------+
only showing top 5 rows
+--------------------+-------+
|            features|Outcome|
+--------------------+-------+
|[148.0,72.0,33.6,...|      1|
|[85.0,66.0,26.6,3...|      0|
|[183.0,64.0,23.3,...|      1|
|[89.0,66.0,28.1,2...|      0|
|[137.0,40.0,43.1,...|      1|
+--------------------+-------+
only showing top 5 rows
Training count: 620
Test count: 148
+--------------------+-------+----------+
|            features|Outcome|prediction|
+--------------------+-------+----------+
|[57.0,80.0,32.8,4...|      0|       0.0|
|[67.0,76.0,45.3,4...|      0|       0.0|
|[71.0,48.0,20.4,2...|      0|       0.0|
|[71.0,78.0,33.2,2...|      0|       0.0|
|[72.0,78.0,31.6,3...|      0|       0.0|
|[76.0,60.0,32.8,4...|      0|       0.0|
|[78.0,50.0,31.0,2...|      1|       0.0|
|[78.0,88.0,36.9,2...|      0|       0.0|
|[84.0,64.0,35.8,2...|      0|       0.0|
|[84.0,82.0,38.2,2...|      0|       0.0|
+--------------------+-------+----------+
only showing top 10 rows
✅ Logistic Regression Accuracy: 0.777027027027027
```

Practical no.08

Aim:- Implement mulitple regression model

Code:-

```
# Install PySpark
!pip install pyspark

# Import Spark
from pyspark.sql import SparkSession

# Create Spark Session
spark = SparkSession.builder.appName('ship_crew_prediction').getOrCreate()

# Load dataset (upload cruise_ship_info.csv first in Colab)
df = spark.read.csv('/content/sample_data/cruise_ship_info.csv', header=True,
inferSchema=True)

df.show(10)
df.printSchema()

# Show columns
print(df.columns)

# Convert categorical column (Cruise_line) to numeric
from pyspark.ml.feature import StringIndexer

indexer = StringIndexer(inputCol='Cruise_line', outputCol='cruise_cat')
df = indexer.fit(df).transform(df)

df.show(5)

# Feature selection
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
   inputCols=[
      'Age',
      'Tonnage',
      'passengers',
      'length',
      'cabins',
      'passenger_density',
      'cruise_cat'
```

```
    ],
    outputCol='features'
)

output = assembler.transform(df)

output.select('features', 'crew').show(5)

# Final dataset
final_data = output.select('features', 'crew')

# Train-test split
train_data, test_data = final_data.randomSplit([0.7, 0.3])

print("Train count:", train_data.count())
print("Test count:", test_data.count())

# Linear Regression model
from pyspark.ml.regression import LinearRegression

lr = LinearRegression(featuresCol='features', labelCol='crew')

model = lr.fit(train_data)

# Evaluate model
training_summary = model.evaluate(train_data)

print("R2 Score:", training_summary.r2)
print("RMSE:", training_summary.rootMeanSquaredError)

# Test model
test_features = test_data.select('features')

predictions = model.transform(test_features)

predictions.show()
```

Output:-

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
+-----------+-----------+---+-----------------+----------+------+------+-----------------+----+
|  Ship_name|Cruise_line|Age|          Tonnage|passengers|length|cabins|passenger_density|crew|
+-----------+-----------+---+-----------------+----------+------+------+-----------------+----+
|    Journey|    Azamara|  6|30.276999999999997|      6.94|  5.94|  3.55|            42.64|3.55|
|      Quest|    Azamara|  6|30.276999999999997|      6.94|  5.94|  3.55|            42.64|3.55|
|Celebration|   Carnival| 26|           47.262|     14.86|  7.22|  7.43|             31.8| 6.7|
|   Conquest|   Carnival| 11|            110.0|     29.74|  9.53| 14.88|            36.99|19.1|
|    Destiny|   Carnival| 17|          101.353|     26.42|  8.92| 13.21|            38.36|10.0|
|    Ecstasy|   Carnival| 22|           70.367|     20.52|  8.55|  10.2|            34.29| 9.2|
|    Elation|   Carnival| 15|           70.367|     20.52|  8.55|  10.2|            34.29| 9.2|
|    Fantasy|   Carnival| 23|           70.367|     20.56|  8.55| 10.22|            34.23| 9.2|
|Fascination|   Carnival| 19|           70.367|     20.52|  8.55|  10.2|            34.29| 9.2|
|    Freedom|   Carnival|  6|110.23899999999999|      37.0|  9.51| 14.87|            29.79|11.5|
+-----------+-----------+---+-----------------+----------+------+------+-----------------+----+
only showing top 10 rows
root
 |-- Ship_name: string (nullable = true)
 |-- Cruise_line: string (nullable = true)
 |-- Age: integer (nullable = true)
 |-- Tonnage: double (nullable = true)
 |-- passengers: double (nullable = true)
 |-- length: double (nullable = true)
 |-- cabins: double (nullable = true)
 |-- passenger_density: double (nullable = true)
 |-- crew: double (nullable = true)

['Ship_name', 'Cruise_line', 'Age', 'Tonnage', 'passengers', 'length', 'cabins', 'passenger_density', 'crew']
+-----------+-----------+---+-----------------+----------+------+------+-----------------+----+----------+
|  Ship_name|Cruise_line|Age|          Tonnage|passengers|length|cabins|passenger_density|crew|cruise_cat|
+-----------+-----------+---+-----------------+----------+------+------+-----------------+----+----------+
|    Journey|    Azamara|  6|30.276999999999997|      6.94|  5.94|  3.55|            42.64|3.55|      16.0|
|      Quest|    Azamara|  6|30.276999999999997|      6.94|  5.94|  3.55|            42.64|3.55|      16.0|
|Celebration|   Carnival| 26|           47.262|     14.86|  7.22|  7.43|             31.8| 6.7|       1.0|
|   Conquest|   Carnival| 11|            110.0|     29.74|  9.53| 14.88|            36.99|19.1|       1.0|
|    Destiny|   Carnival| 17|          101.353|     26.42|  8.92| 13.21|            38.36|10.0|       1.0|
+-----------+-----------+---+-----------------+----------+------+------+-----------------+----+----------+
only showing top 5 rows
+--------------------+----+
|            features|crew|
+--------------------+----+
|[6.0,30.276999999...|3.55|
|[6.0,30.276999999...|3.55|
|[26.0,47.262,14.8...| 6.7|
|[11.0,110.0,29.74...|19.1|
|[17.0,101.353,26....|10.0|
+--------------------+----+
only showing top 5 rows
Train count: 119
Test count: 39
R2 Score: 0.9515218036705976
RMSE: 0.7915127322245725
```

```
+--------------------+------------------+
|            features|        prediction|
+--------------------+------------------+
|[5.0,115.0,35.74,...|11.647198583614239|
|[5.0,122.0,28.5,1...| 6.492936881242284|
|[6.0,90.0,20.0,9....|10.220385715843992|
|[6.0,93.0,23.94,9...|10.570204579930877|
|[6.0,110.23899999...|10.828775413853961|
|[6.0,112.0,38.0,9...|11.032544257913745|
|[7.0,89.6,25.5,9....|11.080568295054155|
|[8.0,91.0,22.44,9...|10.130194321959578|
|[9.0,88.5,21.24,9...| 9.587255205325205|
|[9.0,110.0,29.74,...|11.962898855696157|
|[10.0,68.0,10.8,7...| 6.710490833687548|
|[10.0,81.76899999...| 8.876657375714577|
|[10.0,90.09,25.01...| 8.840082732793523|
|[10.0,138.0,31.14...|12.903640103792956|
|[11.0,108.977,26....|11.020969362303658|
|[11.0,110.0,29.74...| 11.94699690976513|
|[11.0,138.0,31.14...|12.889087021146153|
|[12.0,58.6,15.66,...| 7.463749610658301|
|[13.0,76.0,18.74,...| 8.804213221023582|
|[13.0,85.619,21.1...| 9.674599165338636|
+--------------------+------------------+
only showing top 20 rows
```

Practical no.09

Aim:- Implement movie - review text classification using BI-LSTM

Code:-

```
# ==========================
# 1. INSTALL / IMPORTS
# ==========================
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing import sequence
from tensorflow.keras.layers import Dense, Embedding, LSTM, Bidirectional, Dropout
from tensorflow.keras.datasets import imdb

# ==========================
# 2. LOAD DATASET
# ==========================
num_words = 10000

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=num_words)

print("Train shape:", x_train.shape)
print("Test shape:", x_test.shape)

# ==========================
# 3. PADDING (FEATURE EXTRACTION)
# ==========================
max_len = 200

x_train = sequence.pad_sequences(x_train, maxlen=max_len)
x_test = sequence.pad_sequences(x_test, maxlen=max_len)

y_train = np.array(y_train)
y_test = np.array(y_test)

print("After padding:")
print(x_train.shape, y_train.shape)
print(x_test.shape, y_test.shape)

# ==========================
# 4. BUILD BiLSTM MODEL
# ==========================
```

```
model = Sequential()

model.add(Embedding(input_dim=num_words, output_dim=128, input_length=max_len))
model.add(Bidirectional(LSTM(64)))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

model.summary()

# ==========================
# 5. TRAIN MODEL
# ==========================
batch_size = 250
epochs = 12

history = model.fit(
    x_train, y_train,
    validation_data=(x_test, y_test),
    batch_size=batch_size,
    epochs=epochs,
    verbose=1
)

# ==========================
# 6. RESULTS
# ==========================
print("\nFinal Training Loss:", history.history['loss'][-1])
print("Final Training Accuracy:", history.history['accuracy'][-1])

print("\nFinal Validation Loss:", history.history['val_loss'][-1])
print("Final Validation Accuracy:", history.history['val_accuracy'][-1])

# ==========================
# 7. PLOT RESULTS
# ==========================
plt.figure(figsize=(10,5))

plt.plot(history.history['loss'], label='Training Loss')
```

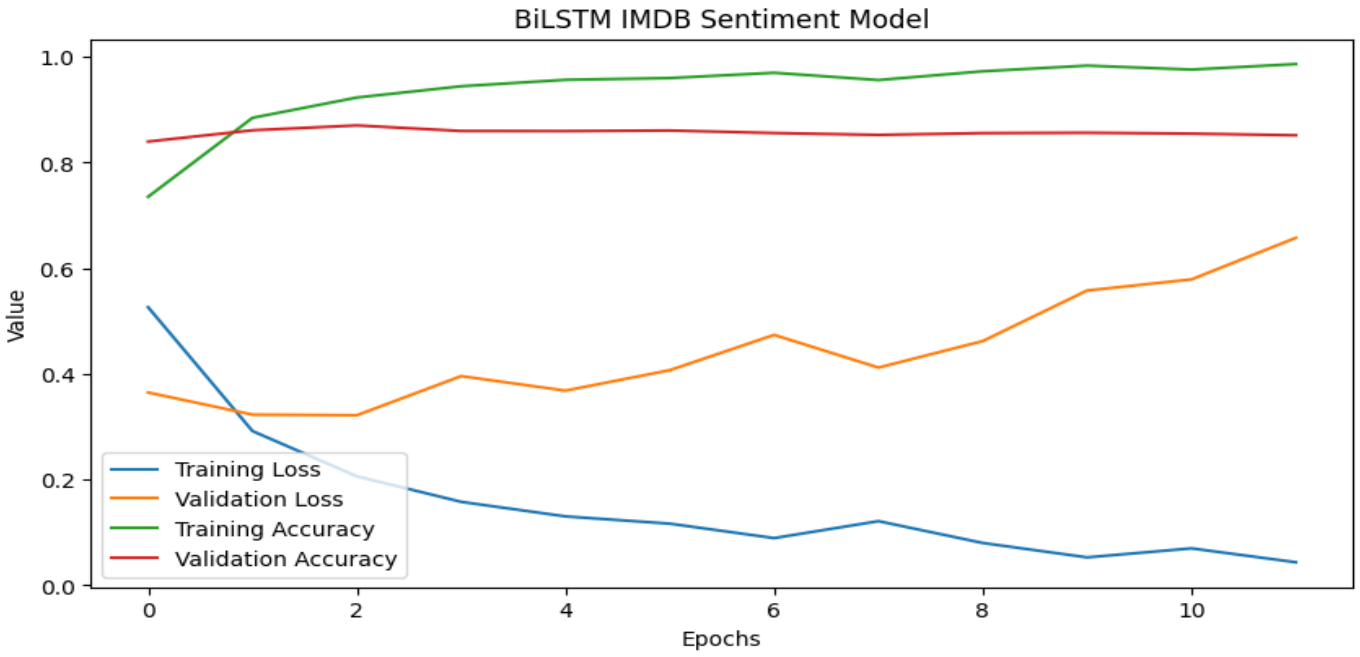
```
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("BiLSTM IMDB Sentiment Model")
plt.xlabel("Epochs")
plt.ylabel("Value")
plt.legend()
plt.show()

# ============================
# 8. SIMPLE PREDICTION EXAMPLE
# ============================
sample = x_test[0].reshape(1, max_len)
prediction = model.predict(sample)

print("\nPrediction score:", prediction[0][0])
print("Sentiment:", "Positive" if prediction[0][0] > 0.5 else "Negative")
```

Output:-

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Train shape: (25000,)
Test shape: (25000,)
After padding:
(25000, 200) (25000,)
(25000, 200) (25000,)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
bidirectional (Bidirectional)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Epoch 1/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 192s 2s/step - accuracy: 0.7354 - loss: 0.5263 - val_accuracy: 0.8399 - val_loss: 0.3650
Epoch 2/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 186s 2s/step - accuracy: 0.8848 - loss: 0.2919 - val_accuracy: 0.8612 - val_loss: 0.3230
Epoch 3/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 171s 2s/step - accuracy: 0.9232 - loss: 0.2063 - val_accuracy: 0.8704 - val_loss: 0.3219
Epoch 4/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 221s 2s/step - accuracy: 0.9446 - loss: 0.1580 - val_accuracy: 0.8599 - val_loss: 0.3960
Epoch 5/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 168s 2s/step - accuracy: 0.9567 - loss: 0.1305 - val_accuracy: 0.8597 - val_loss: 0.3685
Epoch 6/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 166s 2s/step - accuracy: 0.9601 - loss: 0.1167 - val_accuracy: 0.8608 - val_loss: 0.4072
Epoch 7/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 208s 2s/step - accuracy: 0.9700 - loss: 0.0894 - val_accuracy: 0.8560 - val_loss: 0.4740
Epoch 8/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 177s 2s/step - accuracy: 0.9563 - loss: 0.1215 - val_accuracy: 0.8525 - val_loss: 0.4121
Epoch 9/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 207s 2s/step - accuracy: 0.9729 - loss: 0.0802 - val_accuracy: 0.8558 - val_loss: 0.4623
Epoch 10/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 167s 2s/step - accuracy: 0.9836 - loss: 0.0529 - val_accuracy: 0.8565 - val_loss: 0.5580
Epoch 11/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 207s 2s/step - accuracy: 0.9761 - loss: 0.0700 - val_accuracy: 0.8549 - val_loss: 0.5790
Epoch 12/12
100/100 ━━━━━━━━━━━━━━━━━━━━ 209s 2s/step - accuracy: 0.9866 - loss: 0.0437 - val_accuracy: 0.8518 - val_loss: 0.6576

Final Training Loss: 0.04368765652179718
Final Training Accuracy: 0.9865599870681763

Final Validation Loss: 0.6575998663902283
Final Validation Accuracy: 0.8517600297927856
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 420ms/step

Prediction score: 0.00842419
Sentiment: Negative
```

Practical no.10

Aim:- Implement App_User_Segmentation using clustering method.

Code:-

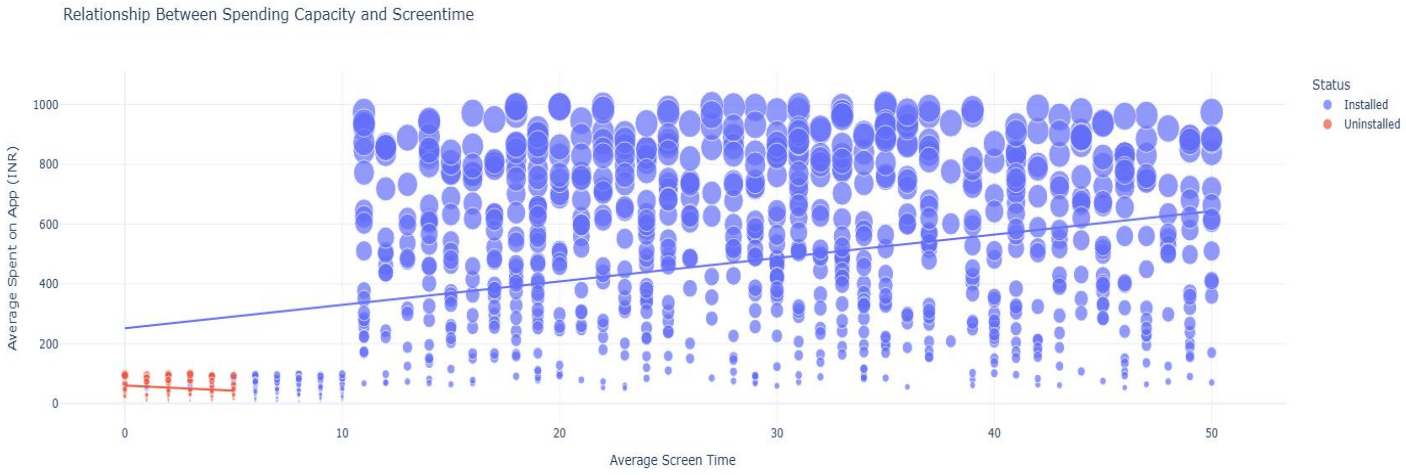
```
import plotly.graph_objects as go
import plotly.express as px
import plotly.io as pio
import pandas as pd

from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans

pio.templates.default = "plotly_white"

data = pd.read_csv("/content/sample_data/userbehaviour.csv")
print(data.head())

print(f'Average Screen Time = {data["Average Screen Time"].mean()}')
print(f'Highest Screen Time = {data["Average Screen Time"].max()}')
print(f'Lowest Screen Time = {data["Average Screen Time"].min()}')

print(data.describe())

print(f'Average Spend of the Users = {data["Average Spent on App (INR)"].mean()}')
print(f'Highest Spend of the Users = {data["Average Spent on App (INR)"].max()}')
print(f'Lowest Spend of the Users = {data["Average Spent on App (INR)"].min()}')

figure = px.scatter(
    data_frame=data,
    x="Average Screen Time",
    y="Average Spent on App (INR)",
    size="Average Spent on App (INR)",
    color="Status",
    title="Relationship Between Spending Capacity and Screentime",
    trendline="ols"
)
figure.show()

figure = px.scatter(
    data_frame=data,
    x="Average Screen Time",
    y="Ratings",
    size="Ratings",
```

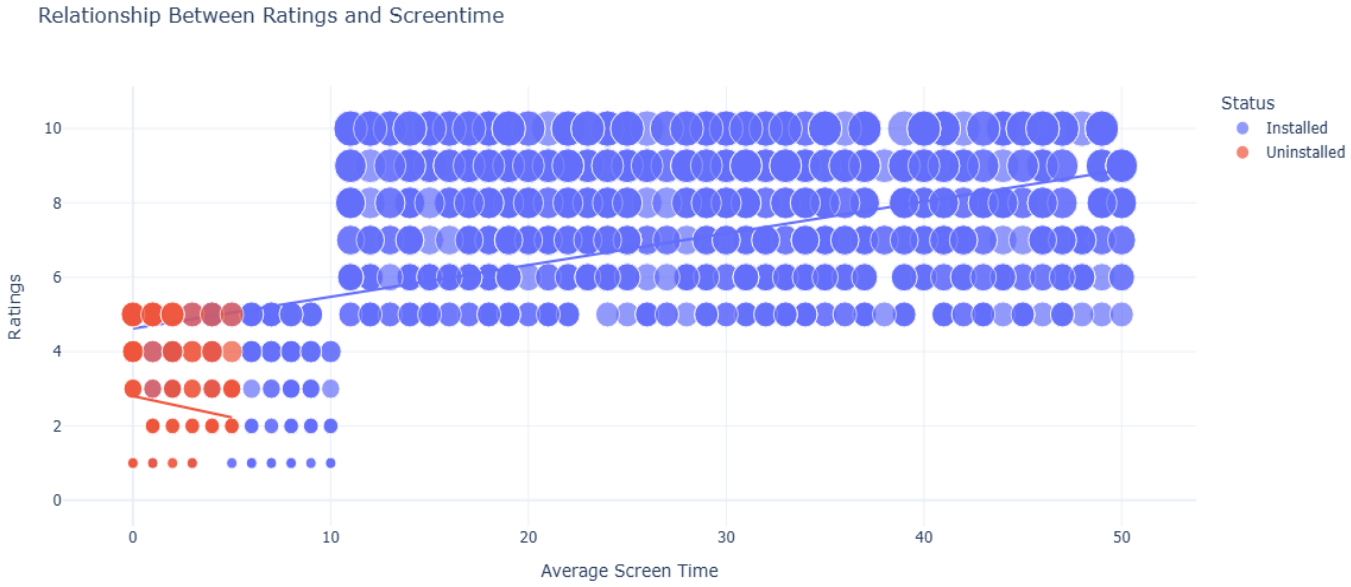
```
    color="Status",
    title="Relationship Between Ratings and Screentime",
    trendline="ols"
)
figure.show()

clustering_data = data[[
    "Average Screen Time",
    "Left Review",
    "Ratings",
    "Last Visited Minutes",
    "Average Spent on App (INR)",
    "New Password Request"
]]

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(clustering_data)

kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
clusters = kmeans.fit_predict(scaled_data)

data["Segments"] = clusters

print(data.head(10))
print(data["Segments"].value_counts())

data["Segments"] = data["Segments"].map({
    0: "Retained",
    1: "Churn",
    2: "Needs Attention"
})

print(data.head(10))

PLOT = go.Figure()

for i in list(data["Segments"].unique()):
    PLOT.add_trace(go.Scatter(
        x=data[data["Segments"] == i]['Last Visited Minutes'],
        y=data[data["Segments"] == i]['Average Spent on App (INR)'],
        mode='markers',
        marker=dict(size=6, line=dict(width=1)),
        name=str(i)
    ))
```

```
PLOT.update_traces(
    hovertemplate='Last Visited Minutes: %{x}<br>Average Spent on App (INR): %{y}'
)

PLOT.update_layout(
    width=800,
    height=800,
    autosize=True,
    showlegend=True,
    yaxis_title='Average Spent on App (INR)',
    xaxis_title='Last Visited Minutes'
)

PLOT.show()
```

Output:-

```
   userid  Average Screen Time  Average Spent on App (INR)  Left Review  \
0    1001                 17.0                       634.0           1
1    1002                  0.0                        54.0           0
2    1003                 37.0                       207.0           0
3    1004                 32.0                       445.0           1
4    1005                 45.0                       427.0           1

   Ratings  New Password Request  Last Visited Minutes       Status
0        9                     7                  2990    Installed
1        4                     8                 24008  Uninstalled
2        8                     5                   971    Installed
3        6                     2                   799    Installed
4        5                     6                  3668    Installed
Average Screen Time = 24.39039039039039
Highest Screen Time = 50.0
Lowest Screen Time = 0.0
            userid  Average Screen Time  Average Spent on App (INR)  \
count   999.000000           999.000000                  999.000000
mean   1500.000000            24.390390                  424.415415
std     288.530761            14.235415                  312.365695
min    1001.000000             0.000000                    0.000000
25%    1250.500000            12.000000                   96.000000
50%    1500.000000            24.000000                  394.000000
75%    1749.500000            36.000000                  717.500000
max    1999.000000            50.000000                  998.000000

       Left Review     Ratings  New Password Request  Last Visited Minutes
count   999.000000  999.000000            999.000000            999.000000
mean      0.497497    6.513514              4.941942           5110.898899
std       0.500244    2.701511              2.784626           8592.036516
min       0.000000    0.000000              1.000000            201.000000
25%       0.000000    5.000000              3.000000           1495.500000
50%       0.000000    7.000000              5.000000           2865.000000
75%       1.000000    9.000000              7.000000           4198.000000
max       1.000000   10.000000             15.000000          49715.000000
Average Spend of the Users = 424.4154154154154
Highest Spend of the Users = 998.0
Lowest Spend of the Users = 0.0
```


Relationship Between Spending Capacity and Screentime
Average Spent on App (INR)
1000
800
600
400
200
0
0
10
20
30
40
50
Average Screen Time
Status
Installed
Uninstalled


Relationship Between Ratings and Screentime
Ratings
10
8
6
4
2
0
0
10
20
30
40
50
Average Screen Time
Status
Installed
Uninstalled